

Assignment 2

Team number: 22

Team members

Name	Student Nr.	Email
Darian Janevski	2728049	d.janevski@student.vu.nl
Diar Kamberi	2738305	d.kamberi@student.vu.nl
Kosta Bakalinov	2741568	k.bakalinov@student.vu.nl
Ibrahim Abdelraouf Ibrahim Badr	2737268	ibrahimabdelraoufibrabimbadr@student.vu.nl

Summary of changes from Assignment 1

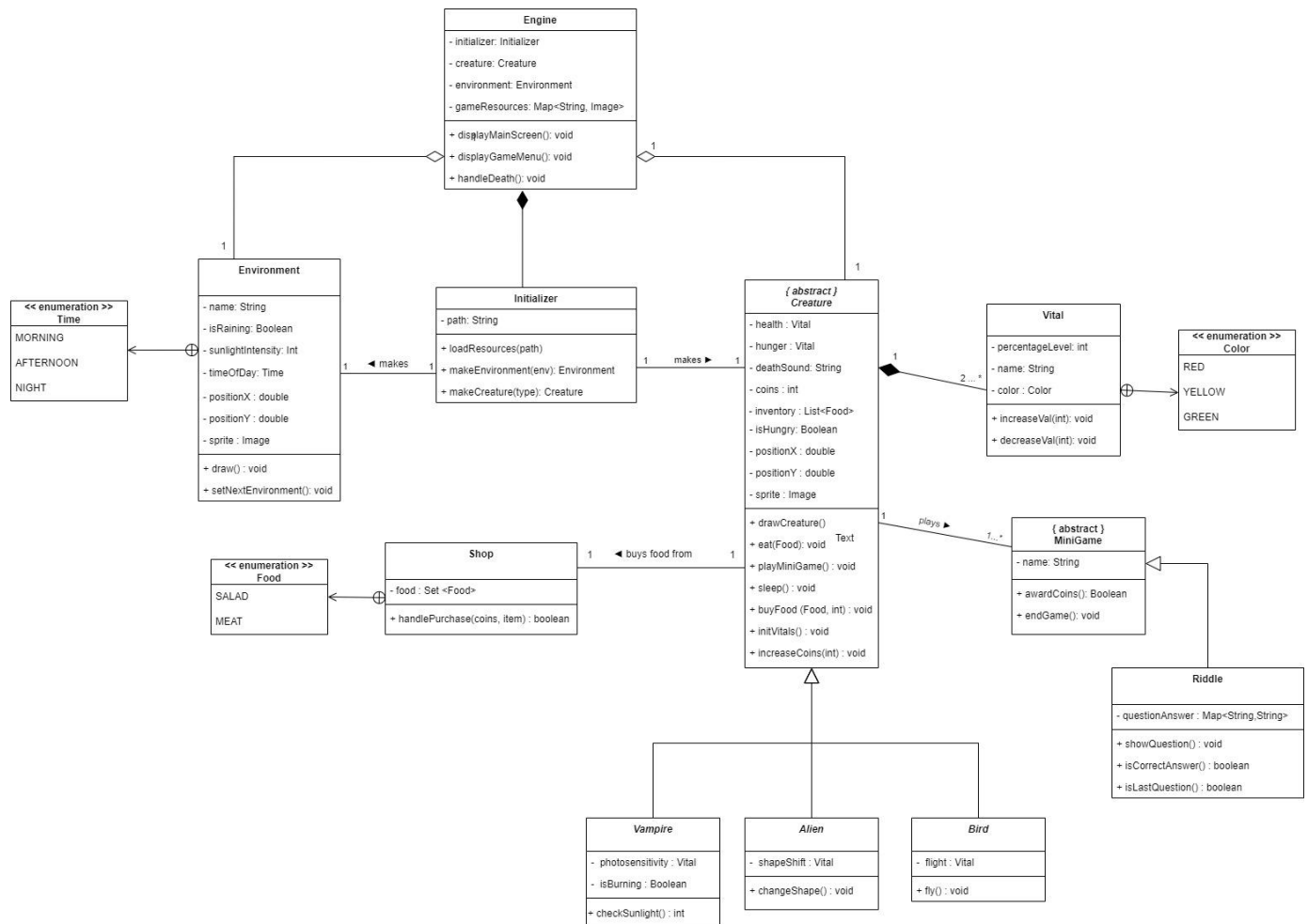
Author(s): Darian Janevski, Diar Kamberi, Kosta Bakalinov, and Ibrahim A. I. Badr

After the meeting with the mentor we discussed our project and realized what we needed to change. The following list depicts all of the changes we made to our project:

- The creatures from which the player can choose are: a Bird, Alien, and Vampire.
- We revamped the vital system by removing a lot of complexity. Now every creature will have three vital bars. Two of them are shared amongst every creature, while the third one is unique to the pet. The shared ones are *health* and *hunger*. The unique ones are: shapeshifting for the alien, photosensitivity meter for the vampire, and flight for the bird.
- The environment system is also simplified. There are forest, desert, and Antarctica settings from which the user can choose. The only variable that is environment dependent is the sunlight intensity. This will have an effect on the Vampire creature if chosen. There won't be other temperature or climate attributes, as the *mood* vital is also removed.
- The handling of the creature's death is established. It dies once its health bar reaches zero. The player can prevent that from happening in many ways. In case they don't, a game-over message will pop up and they would need to watch an ad to get a chance to revive the pet.

Class diagram

Author(s): Darian Janevski, Diar Kamberi, Kosta Bakalinov, and Ibrahim A. I. Badr



The image was large, so to see it better please follow the link shown below.

Link:

<https://app.diagrams.net/?src=about#GlcsKtUUxNWEGOsYZDyGrYjQNrD9QsQIyU>

Description:

The class diagram depicted above models the class structure and relationships between the classes in our “Vivarium” game design. In our design, the **Engine** class is responsible for managing the graphics and logic of our game. It will contain only one instance of **Initializer**, **Creature**, and **Environment**. Because of how our game works, it does not make sense to have multiple creatures or multiple environments. The **Initializer** class is used by the **Engine** class to load the gameResources and to create the **Creature** that the user chooses which can be of type **Bird**, **Vampire**, **Alien**, or other types which can be added. The **Creature** class is an abstract class as we do not want to have a generic creature that can be instantiated. The **initializer** class is in a composite relationship with the **Engine** class because we want the only instance of **Initializer** to be used by the **Engine** and we do not want to have an instance of **Initializer** out

of the **Engine** class. The **Creature** and **Environment** classes are in an aggregation relationship with the **Engine** class because the game logic is handled by the **Engine**, but logically they can exist even if the **Engine** class is destroyed. The **Vital** class is used to represent the vitals of the creature, so logically a **Vital** instance cannot exist without a creature.

Our model allows for the extensibility of the **Creature**, **Vital** and **Minigame** classes. To add a new **Creature** type, you extend the **Creature** class which contains the basic attributes and vitals, and add the specific methods and attributes in the newly created subclass. As can be seen from the class diagram, **Bird**, **Alien**, and **Vampire** inherit from the **Creature** and include their own **Vital** and methods. The vitals of a creature can also be extended by making an instance of the **Vital** you want to include, and adding the method that handles the logic of the new **Vital**. The **Minigame** can also be extended by the minigame you want to add, after you handle the logic of the game you want to add. The added **Minigame** has minimal influence on the system since it is contained under the **Minigame** and should handle the added game logic in itself, separately from the **Engine**.

The **Shop** and the **Minigame** classes represent extra features of our game, where we want the creature to buy food from a **Shop**, which the food is then eaten by the creature to increase the *hunger percentageLevel*. The **Minigame** is an abstract class from which all minigames that we add should inherit from, and a **Creature** is in a one to many relationship with **Minigame** since we want to be able to play many minigames with one **Creature**. It contains the awardCoins(), and endGame() methods which should be defined by every class that inherits from it. For now, we have the **Riddle** game which contains a questionAnswer map that is used to validate answers.

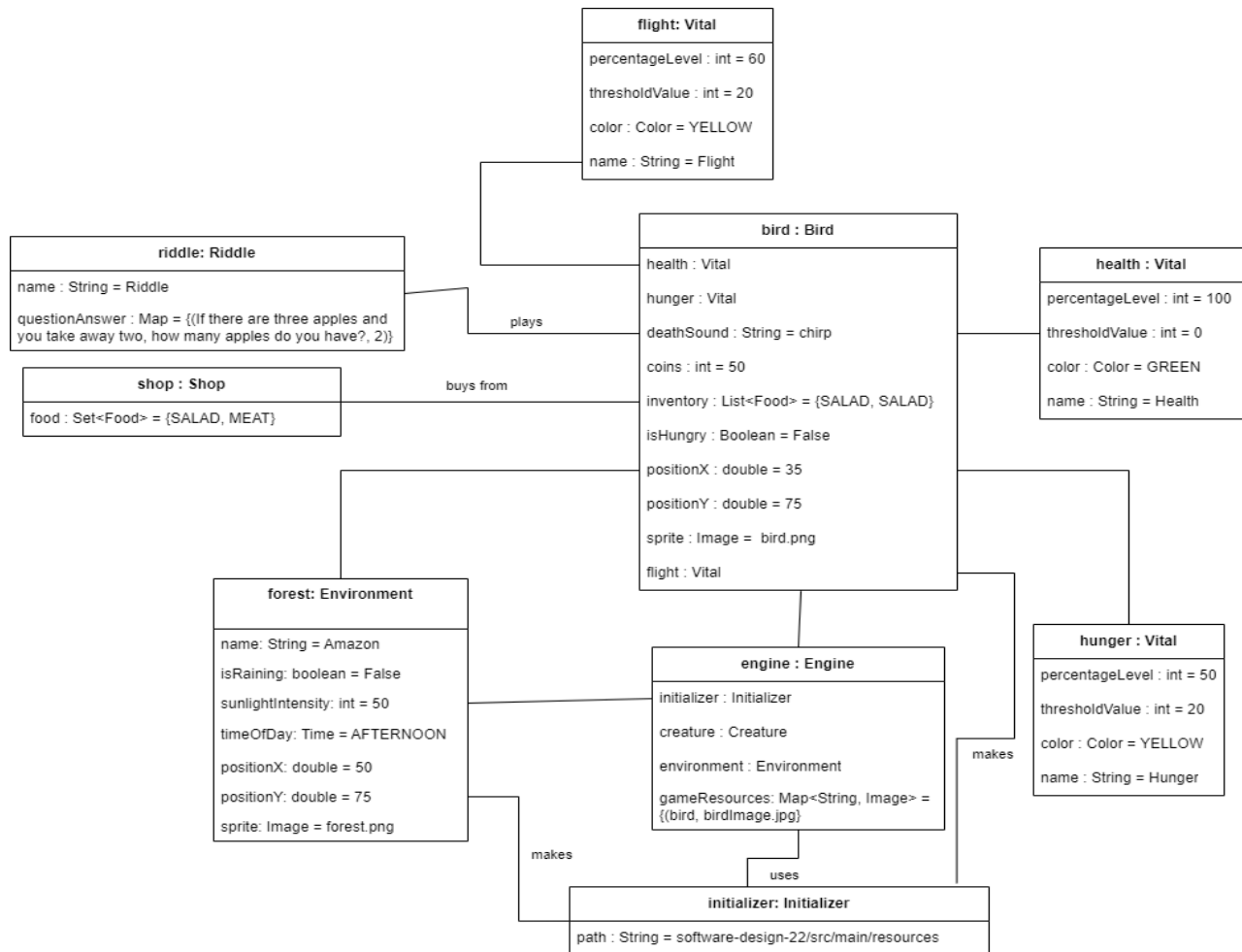
We have **Time**, **Color** and **Food** as enumeration classes to allow for extra extensibility to the game entities. **Time** represents the timeOfDay of the **Environment** which can be advanced by the setNextTimeOfDay() function which should advance it to the next logical step in time phase (morning to afternoon, afternoon to night, night to morning). **Color** allows to add different colors to the vital bars which will be expressed visually on the screen, **Food** will allow to add different foods (if you want Pasta, Lasagna etc.).

An alternative design we considered was making the **Vital** class an abstract class from which other vitals would inherit from. However, we realized that modelling our system in such a way would create unnecessary classes where they did not need any attributes or methods. So, now you can instantiate the **Vital** class with a name, color, and percentageLevel and handle the logic in the actual creature.

Another alternative design we considered was to put all the game logic in the creature. So, the updating of the states, displaying the images and every logic would be done in the **Creature**. We realized that this design does not make much sense and would result in our **Creature** class having methods and attributes which do not essentially belong to a creature. So, we decided to include an **Engine** class which will act as the game logic class of our game.

Object diagram

Author(s): Darian Janevski, Diar Kamberi, Kosta Bakalinov, and Ibrahim A. I. Badr



Link:

<https://app.diagrams.net/?src=about#G1flm3l5n6l9ATXDbD9FgOmEMhnTST2Y3e>

Description:

The object diagram above presents a snapshot of our system at a particular point in time. In this scenario, the player has chosen the bird creature as their game character and the forest as their environment. As a result, the *initializer* has created the *forest* and *bird* objects. The *engine* object is the one initiating the action of creating the *bird* and *forest* objects. *Initializer* also has a game resources hash-map attribute, where all of the images of the relevant components are stored. In our diagram an example key-value pair is (bird, birdImage.jpg).

The *bird* object is at the center of our object diagram. It is a subclass of **Bird** and **Creature**, thus it inherits all the attributes from its parents and it also conforms to the relationships in the class diagram. One unique attribute that this object has is the *flight* vital which separates it from the other creature objects.

Health, *flight*, and *hunger* objects are all children of the **Vital** class, and they are associated with the *bird*. From the *vitals*' percentages and colors it is evident that at the current state, the bird's *health* is at maximum, while hunger and flight meter are somewhat alright.

The *riddle* object is the mini-game that the player can interact with. It has a hash-map attribute that stores all of the riddles and answers. In our object diagram design, we decided to only include one riddle example in the map to make the diagram readable and give an idea of a typical question.

There are other **Environment** and **Creature** objects in the game code, however it is our design decision to show only the objects that are chosen by the user and created at runtime.

2% every minute. Another way it can break from the *Hungry* state is by transitioning to the *Death* state, which occurs when the health percentage is zero. On the other hand, If the “eat” event was triggered before the *health* percentage reached zero, the *hunger* level will increase depending on the food type.

When the “fly” event is prompted, the bird changes its state to the *Flying* state. After 3 seconds, it goes back to *Idle State*, and as a result the bird’s *health* increases while the hunger decreases.

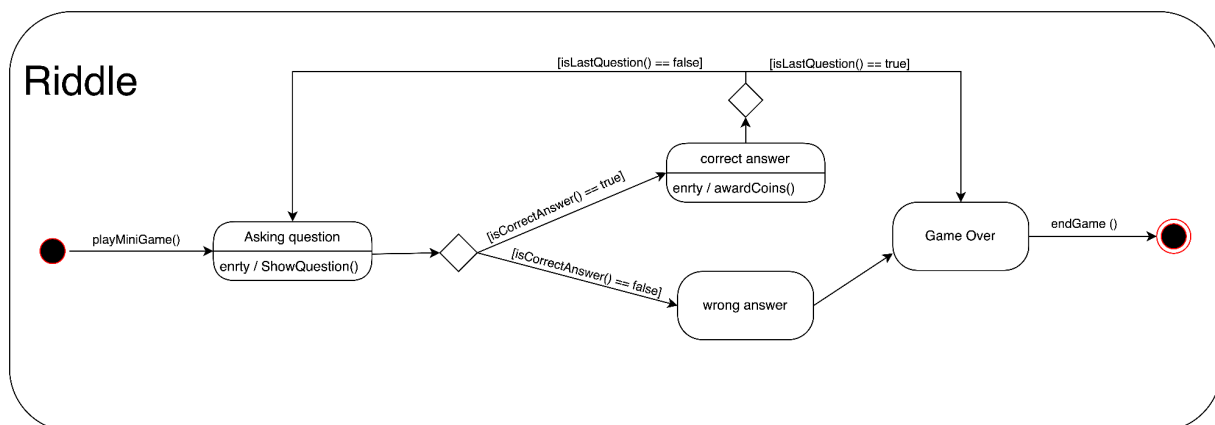
If the player decides to play the mini-game, the state is changed to *Playing*. The state can be exited if the user either successfully completes the game or fails to answer some question. Based on the result the bird might be awarded coins.

Finally, only when it’s nighttime in the game, the user would be able to put the bird in a *Sleeping* state. This state can only be exited in the morning time in the game, and as a result, the health and flight percentages increase while the hunger level decreases.

As stated in the paragraph above, the bird creature can only sleep at night and wake up in the morning. We did this intentionally because we want in our game different creatures to have different sleeping patterns. For example, the vampire creature would only be able to sleep during the day and be awake at night.

When developing the game, we considered an alternative approach for handling the hunger bar of the creature. Our strategy was to increase or decrease the *hunger* percentage by a random number each time. However, we took into account our main stakeholders and decided that this mechanism might be too complicated and problematic for them.

Riddle mini-game class:



Link (for the two-state machine diagrams):

https://drive.google.com/file/d/1pTCV7i2G35DBClA5haOy5xaa_UenBMSh/view?usp=sharing

Description:

The second state machine diagram illustrates the state transitions for the **Riddle** mini-game class during our game runtime. The player can choose to play a mini-game to get more coins so as to buy food for their creature. First, The game begins with the “*Asking question*” state, where a question is displayed on the player’s screen.

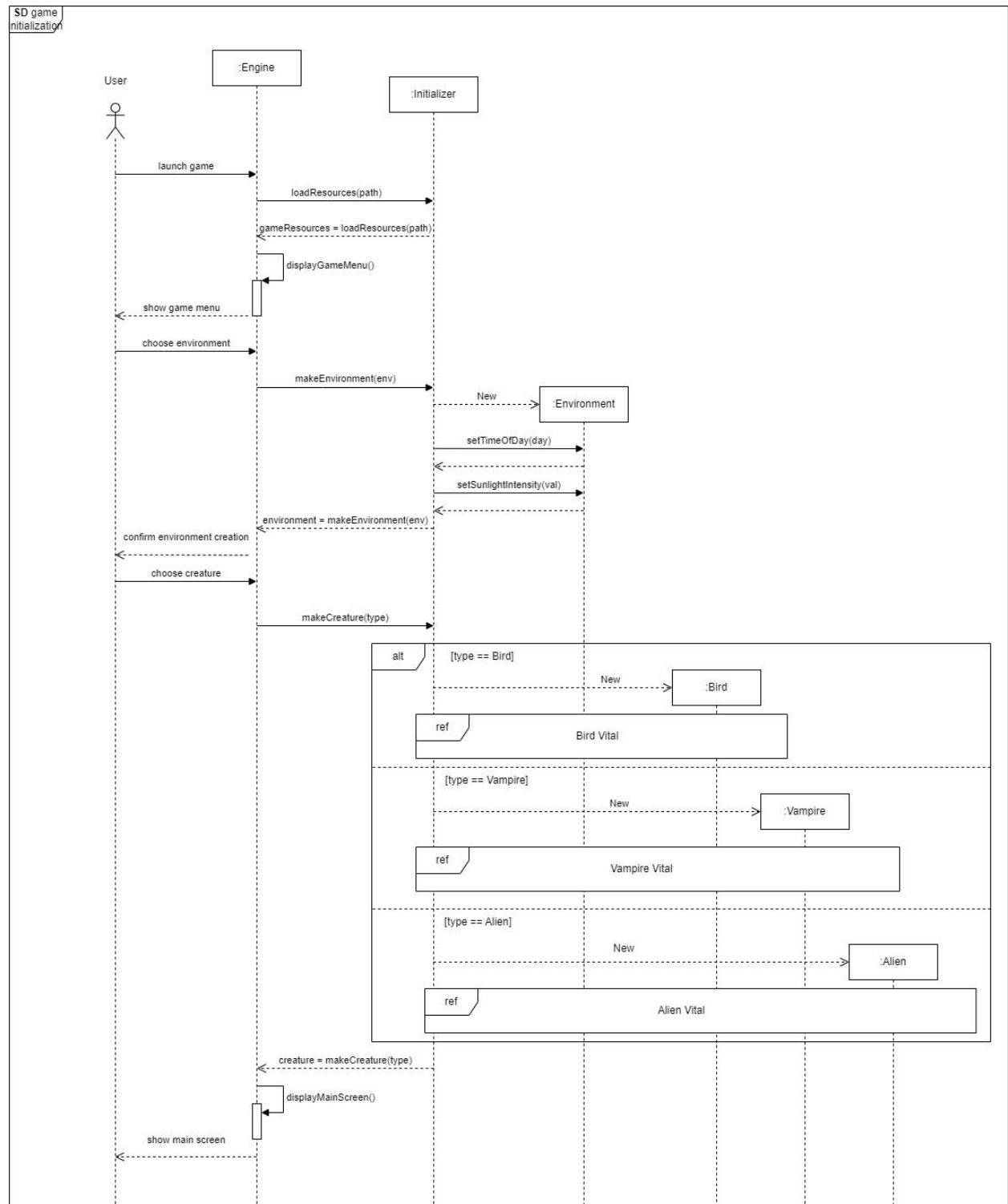
According to the answer of the player the game state will change. Thus, if the player answered the question correctly, they will be awarded coins and either move to the next question, if there are questions left, or to the “*Game-over*” state where the player returns back to the main game. On the other hand, if the user answered the question incorrectly, the “*Game-over*” state will be triggered.

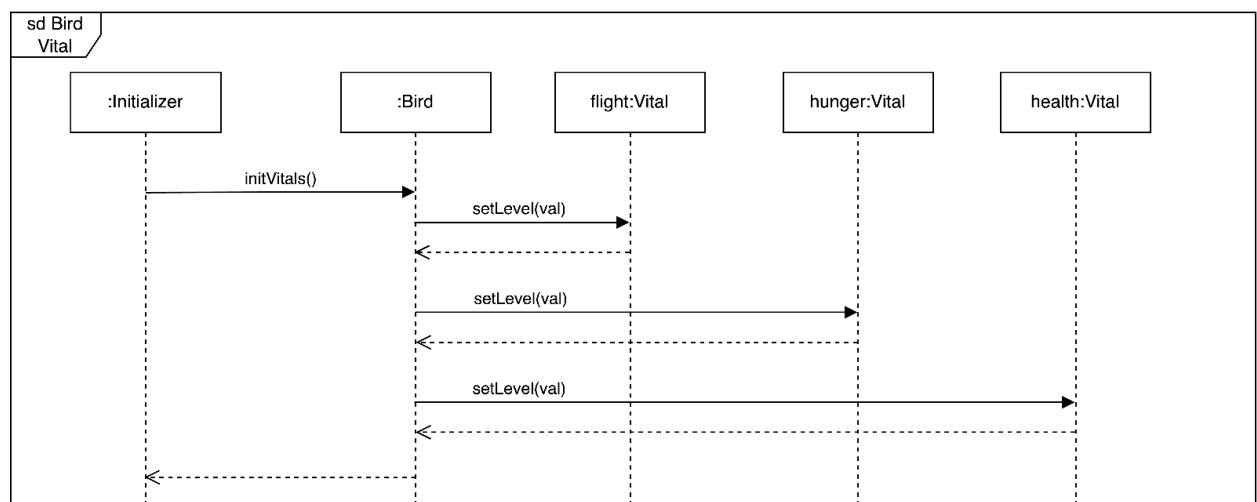
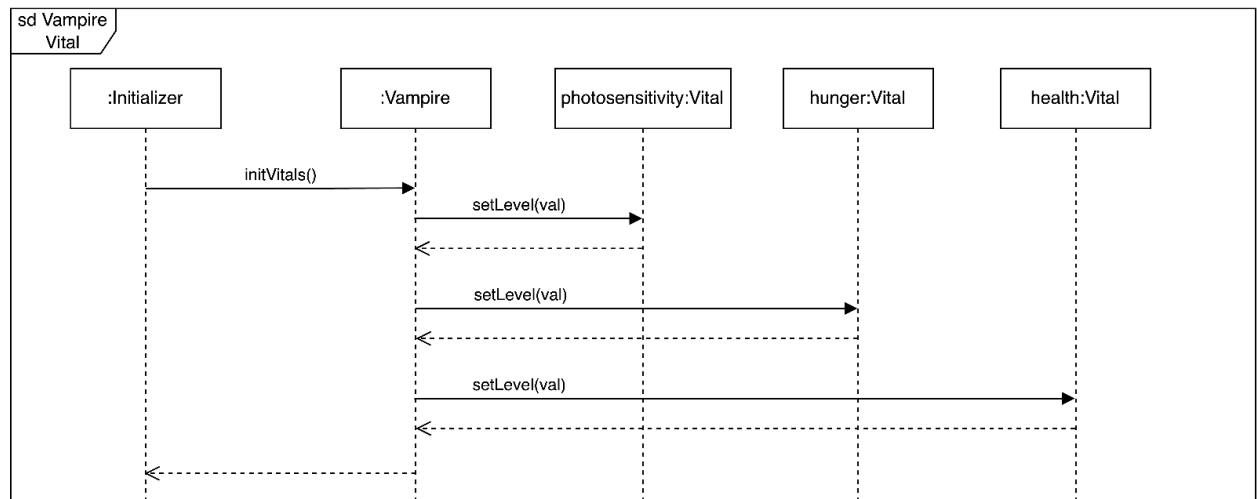
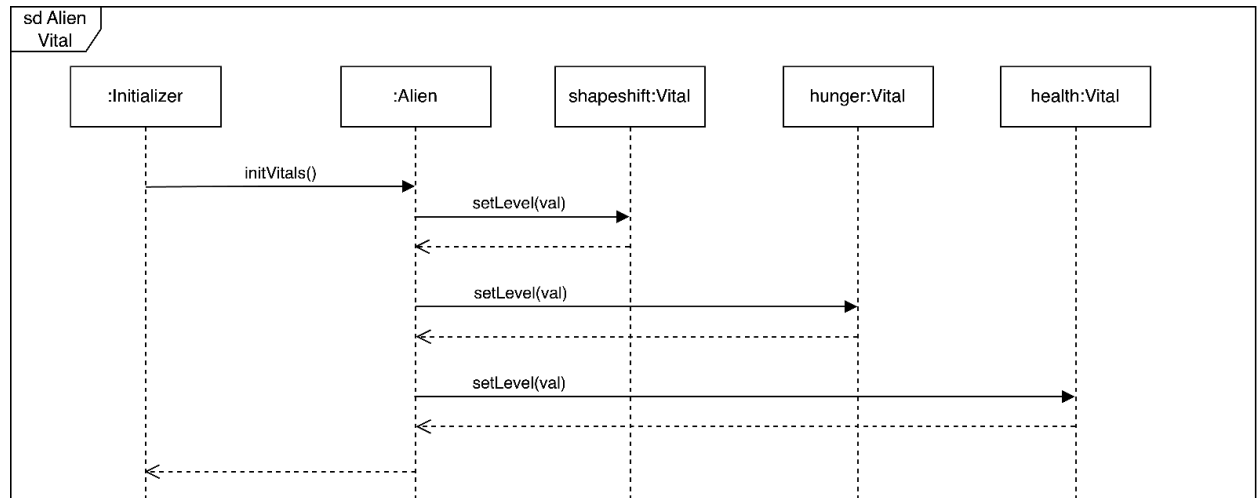
In our initial game design, players are required to type their answers in an input box, which may seem tedious. Moreover, players only have one chance to answer the question wrongly. Limiting young players’ opportunities to learn and improve. To address these issues, an alternative design for the game can consist of different question types such as multiple choice or matching. Additionally, players can have a few attempts to answer incorrectly which will make the game more engaging and interesting for young players.

Sequence diagrams

Author(s): Darian Janevski, Diar Kamberi, Kosta Bakalinov, and Ibrahim A. I. Badr

Game initialization:





Link(for the two sequence diagrams):

<https://drive.google.com/file/d/1dxBTZLJvZKnVoh9Y7GLTcF3SbSrh8vOv/view?usp=sharing>

Description:

The sequence diagram in the picture above depicts the initialization of the game. What happens when the user launches the game, is how the start screens are shown, the objects are created and the values are set at the beginning.

The game initialization starts with the User actor who launches the game. This is a request sent to the engine that starts the whole process. We decided to show the user as a starting point since he interacts with the game and later on is shown all of the screens where he has to choose the **Environment** and the **Creature** to play the game. This makes it easier to understand what the user will see and what will happen in the background during the initialization.

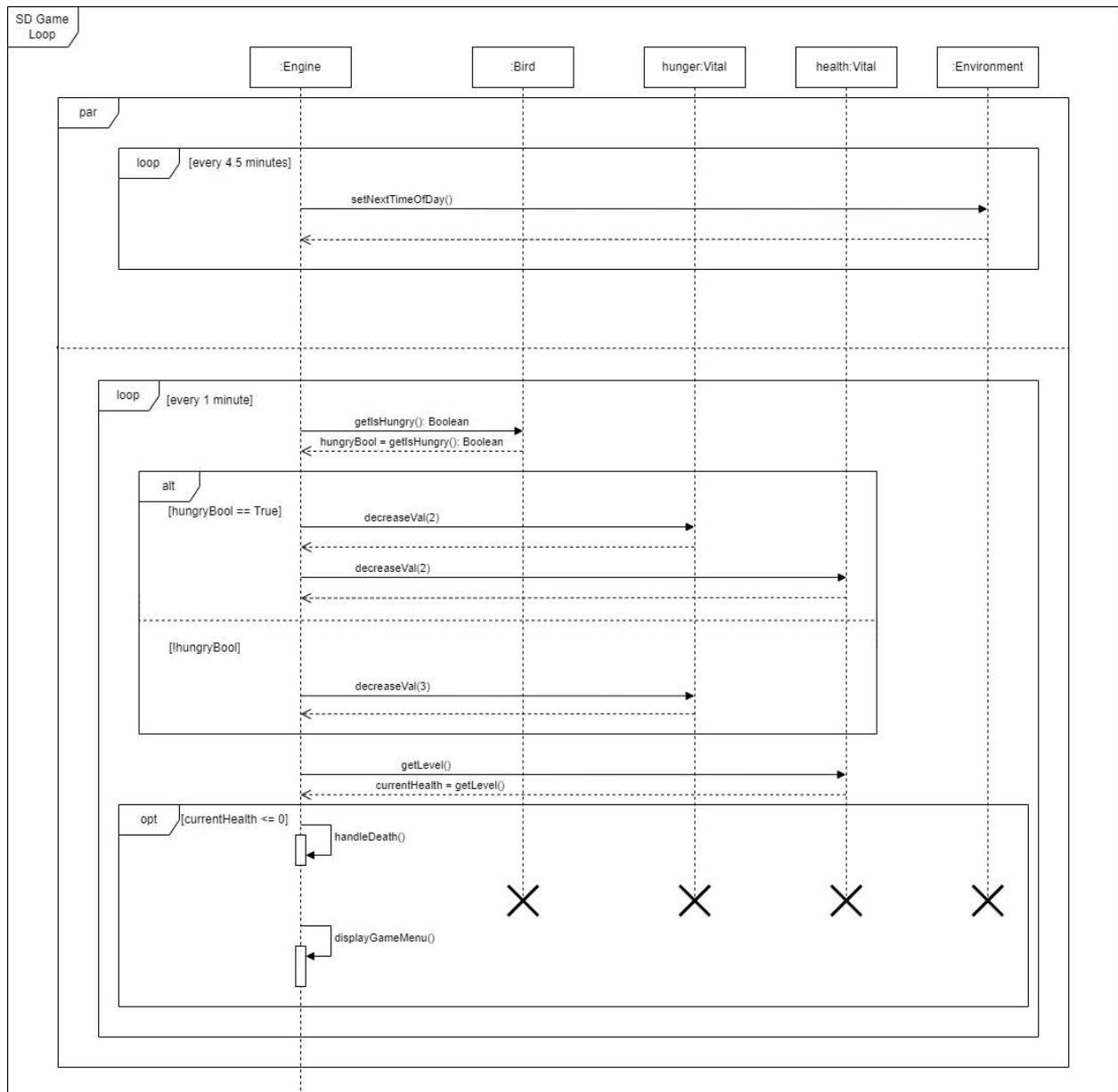
After the user launches the game, the **Engine** will send a message to the **Initializer** to load the files with the path as a parameter, and after the **Initializer** will return them to the **Engine** and store them in the variable gameResources. When this operation returns the **Engine** will make a self-call displayGameMenu which shows the main game menu back to the **User**. From there he can choose the **Environment** and the **Creature**. We use the **Initializer** to load the most recent resources because we might decide to use different images or sizes of the sprites.

The user chooses the **Environment** and from there the **Engine** creates the environment chosen and sets its time of day and the sunlight intensity, it also returns a confirmation to the user. He then chooses a **Creature** from which the **Initializer** creates the chosen creature. Here we have three alternatives for each creature since each creature has one different vital which will be created and set depending on which one is chosen. In all three scenarios, the **Initializer** will create a new **Creature**, the *health* and *hunger* vitals which also will be set with val as a parameter. If the **Creature** is a **Vampire** then a *photosensitivity* vital is created and set, if it is a **Bird** then a *flight* vital is created and set and if it is an Alien then a *shapeshift* vital is created and set.

The **Initializer** returns the **Creature** to the **Engine** which makes a self-call to display the Main menu and that is then shown to the **User**.

An alternative design for the sequence of the gameInitialization is having save files, where you can choose to make a new game or load a saved game. For now, we decided to not include that as it adds complexity to the game, for which we might not have enough time to execute. Another design

Game Loop



Link(for the two sequence diagrams):

<https://drive.google.com/file/d/1dxBTZLJvZKnVoh9Y7GLTcF3SbSrh8vOv/view?usp=sharing>

Description:

The sequence diagram in the picture above depicts the game loop. How the game will run and how the vitals are all updated as time passes. In the game loop, we did not show the user input since this sequence diagram only shows how the game reacts to the states of the creature and the environment.

The sequence diagram depicted in the picture above depicts the general game loop of our “Vivarium” game. It represents how the game will run and how the vitals are all updated as time passes. The game loop does not include the user inputs to “fly”, “sleep”, “eat” “shapeshift” since those are handled outside of the game loop.

We have two loops that will run concurrently, represented by the par frame. The first loop runs every 4.5 minutes and in it the **Engine** will set the next time of day of the **Environment**. While the other loop will run every 1 minute and will check and update the vitals of the **Creature**.

At the beginning of the second loop, the Engine checks if the **Creature** is hungry and stores the boolean in the variable hungryBool. If the boolean is true and the **Creature** is indeed hungry, then the **Engine** will decrease the *hunger* and the *health* vital by 2. While if the boolean is false then the **Creature** is not hungry and the **Engine** will decrease only the *hunger* by 3.

After the two alternatives, the **Engine** will get the health level of the creature, and based on that it will handle the death. If the health value is below or equal to 0 then the Engine makes a self-call and destroys the *creature* and *environment* variables. After that, it shows the GameMenu again where the player can initiate a new game by selecting a creature and environment again (in this scenario the resources are already loaded in the gameResources).

Time logs

Team Number:	22		
Member	Activity	Week number	Hours
-Darian Janevski -Diar Kamberi -Kosta Bakalinov -Ibrahim A. I. Badr	Discussing the feedback from assignment 1	2	2
-Darian Janevski -Diar Kamberi -Kosta Bakalinov -Ibrahim A. I. Badr	Splitting the tasks for assignment 2	3	1
-Ibrahim A. I. Badr	planning and preparing the framework for A2 (docs, requirements, cloud diagrams,...)	3	2

-Darian Janevski -Diar Kamberi -Kosta Bakalinov -Ibrahim A. I. Badr	Designing the core structure of the class diagram.	3	4
-Darian Janevski -Diar Kamberi -Kosta Bakalinov -Ibrahim A. I. Badr"	Mentor meeting	3	1
-Darian Janevski -Diar Kamberi -Kosta Bakalinov -Ibrahim A. I. Badr	Discussing the feedback from the mentor meeting	4	3
-Darian Janevski -Diar Kamberi -Kosta Bakalinov -Ibrahim A. I. Badr"	Implementing the relationships between classes in class diagram	4	4
-Kosta Bakalinov	Designing the core structure of object diagram	4	4
-Ibrahim A. I. Badr	Designing the core structure of the first and second state machine diagram	4	5
-Darian Janevski	Designing the core structure of the first and second sequence diagram	4	5
-Ibrahim A. I. Badr	Designing the core structure of the first and second state machine diagram	4	5
-Diar Kamberi	Adding the attributes and methods for the class diagram.	4	5
-Kosta Bakalinov	Implementing the object diagram	4	3
-Ibrahim A. I. Badr	Implementing the two-state machine diagrams	4	3
-Darian Janevski	Implementing the two sequence diagrams	4	3
-Diar Kamberi	Implementing the base code for the game	4	3
-Darian Janevski -Diar Kamberi -Kosta Bakalinov -Ibrahim A. I. Badr	Discussing the progress of each team member and linking the diagrams together	4	2

-Darian Janevski -Diar Kamberi -Kosta Bakalinov -Ibrahim A. I. Badr"	Mentor meeting	4	1
-Darian Janevski -Diar Kamberi -Kosta Bakalinov -Ibrahim A. I. Badr	Discussing the feedback from the mentor meeting	5	2
-Diar Kamberi	Finalizing the class diagram	5	4
-Kosta Bakalinov	Finalizing the object diagram	5	4
-Ibrahim A. I. Badr	Finalizing the two-state machine diagrams	5	4
-Darian Janevski	Finalizing the two sequence diagrams	5	4
-Darian Janevski -Diar Kamberi -Kosta Bakalinov -Ibrahim A. I. Badr	Checking the progress of all the diagrams and thinking about alternative designs and what could be done better.	5	3
-Darian Janevski -Diar Kamberi -Kosta Bakalinov -Ibrahim A. I. Badr	Mentor meeting	5	1
-Darian Janevski -Diar Kamberi -Kosta Bakalinov -Ibrahim A. I. Badr	Discussing the feedback from the mentor meeting and working on the final submission files	5	6
		TOTAL	84